

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1
«Організація професійного Python-проекту та Git»
з курсу «Професійний Python»
Варіант №1 — «Система обліку студентів»**

Виконав: Лис Артур

Група: ФЕП-31с

Перевірів(ла): _____

Дата: 02.09.2026

Львів — 2026

1. Мета роботи

Навчитися організовувати Python-проект так, як це роблять у професійній розробці: ізольоване середовище (virtual environment), src-layout, поділ коду на пакет і модулі, конфігурація в pyproject.toml, документація в README.md, контроль версій у Git з логічною історією комітів. Практично реалізувати консольний застосунок за індивідуальним варіантом №1 і закласти основу проєкту, який буде розвиватися протягом усього курсу.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Система обліку студентів». Для кожного студента зберігаються ім'я, прізвище, група та середній бал. Потрібно реалізувати додавання студента, виведення списку, пошук за групою, визначення студента з найбільшим середнім балом та обчислення середнього бала групи.

Обов'язкові вимоги до проєкту: src-layout, щонайменше три модулі, окрема модель даних і окремий модуль бізнес-логіки, main.py з конструкцією `if __name__ == "__main__":`, type hints, README.md, .gitignore, pyproject.toml, Git-репозиторій.

Із завдань підвищеної складності реалізовано: введення даних з клавіатури, menu-driven interface, валідацію введення, сортування (рейтинг), пошук за кількома критеріями, статистичні показники, окремі конфігураційні константи, власні винятки, dataclass та README з прикладом запуску.

3. Структура проєкту

```
student_manager/
├── src/
│   └── student_manager/
│       ├── __init__.py
│       ├── cli.py
│       ├── config.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       └── registry.py
├── tests/
│   └── test_registry.py
├── .gitignore
├── pyproject.toml
└── README.md
```

Каталог `src/student_manager` — це сам пакет. Код навмисно поділено дрібніше, ніж у прикладі з методичних вказівок: константи винесено в `config.py`, винятки предметної області — в `exceptions.py`, бізнес-логіка зібрана в класі `StudentRegistry` у `registry.py`, а все, що стосується консолі (таблиці, меню, розбір введення), живе окремо в `cli.py`. `main.py` лише збирає це до купи. Каталог `tests` містить `pytest`-тести бізнес-логіки, які не залежать від консолі.

Такий поділ обрано з думкою про наступні лабораторні роботи: до StudentRegistry можна буде підключити базу даних або REST API, не чіпаючи ні модель, ні меню.

4. Опис створених модулів

4.1. config.py

Єдине місце для «магічних чисел»: межі шкали оцінювання, поріг відмінника та регулярний вираз для назви групи. Решта модулів імпортують ці константи, а не дублюють їх.

```
"""Project-wide constants. The only place where magic numbers are allowed to live."""

GRADE_MIN = 0.0
GRADE_MAX = 100.0
EXCELLENT_THRESHOLD = 90.0

# ФЕР-31с, ПМ-22, КН-31а: 2-5 capital letters, dash, two digits, optional lowercase suffix
GROUP_PATTERN = r"[A-ЯІІЄґA-Z]{2,5}-\d{2}[a-яіієґa-z]?"
```

4.2. exceptions.py

Власна ієрархія винятків. Базовий StudentManagerError дає змогу в меню одним ехсерптом перехопити будь-яку помилку предметної області. Конкретні винятки успадковують ще й стандартні ValueError чи LookupError, тому код, який нічого не знає про проєкт, теж обробить їх коректно.

```
"""Domain-specific exceptions. Callers catch StudentManagerError to handle any of them at once."""

from student_manager.config import GRADE_MAX, GRADE_MIN

class StudentManagerError(Exception):
    """Base class for every error raised by the project."""

class InvalidGradeError(StudentManagerError, ValueError):
    """Average grade outside the 100-point scale."""

    def __init__(self, grade: float) -> None:
        super().__init__(f"середній бал має бути в межах {GRADE_MIN:g}..{GRADE_MAX:g}, отримано {grade!r}")
        self.grade = grade

class InvalidGroupError(StudentManagerError, ValueError):
    """Group name that does not look like ФЕР-31с."""

    def __init__(self, group: str) -> None:
        super().__init__(f"назва групи {group!r} не схожа на ФЕР-31с")
        self.group = group
```

```
class StudentNotFoundError(StudentManagerError, LookupError):
    """A query matched no student."""
```

4.3. models.py

Модель даних — `dataclass Student` зі `slots=True` (менше пам'яті, захист від випадкових атрибутів). У `__post_init__` виконується нормалізація (обрізання пробілів) та валідація: непорожні ім'я і прізвище, назва групи за шаблоном, бал у межах шкали. Некоректний об'єкт просто не може бути створений.

```
"""Data model of the registry."""

from __future__ import annotations

import re
from dataclasses import dataclass

from student_manager.config import EXCELLENT_THRESHOLD, GRADE_MAX, GRADE_MIN, GROUP_PATTERN
from student_manager.exceptions import InvalidGradeError, InvalidGroupError

@dataclass(slots=True)
class Student:
    """One student: name, academic group and average grade on the 100-point scale."""

    first_name: str
    last_name: str
    group: str
    average_grade: float

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name:
            raise ValueError("ім'я та прізвище не можуть бути порожніми")
        if not re.fullmatch(GROUP_PATTERN, self.group):
            raise InvalidGroupError(self.group)
        if not GRADE_MIN <= self.average_grade <= GRADE_MAX:
            raise InvalidGradeError(self.average_grade)

    @property
    def full_name(self) -> str:
        return f"{self.last_name} {self.first_name}"

    @property
    def is_excellent(self) -> bool:
        return self.average_grade >= EXCELLENT_THRESHOLD
```

4.4. registry.py

Уся бізнес-логіка варіанта. `StudentRegistry` зберігає список студентів і відповідає на запити: усі студенти, студенти групи, найкращий (загалом або в групі), середній бал групи,

рейтинг, багатокритеріальний пошук, статистика. Клас реалізує `__len__` та `__iter__`, тому з ним працюють `len()` і цикл `for`. Порівняння назв груп нечутливе до регістру завдяки `casefold()`. Рейтинг спирається на стабільність сортування Python: спочатку за прізвищем, потім за балом, тому однакові бали йдуть за алфавітом.

```
"""Business logic: an in-memory registry of students. Knows nothing about the console."""

from __future__ import annotations

from collections.abc import Iterable, Iterator
from statistics import mean

from student_manager.exceptions import StudentNotFoundError
from student_manager.models import Student


class StudentRegistry:
    """Keeps students and answers questions about them."""

    def __init__(self, students: Iterable[Student] = ()) -> None:
        self._students: list[Student] = []
        for student in students:
            self.add(student)

    def __len__(self) -> int:
        return len(self._students)

    def __iter__(self) -> Iterator[Student]:
        return iter(self._students)

    def add(self, student: Student) -> Student:
        """Register a student; the same person in the same group twice is a mistake."""
        if any(s.full_name == student.full_name and s.group == student.group for s in self._students):
            raise ValueError(f"{student.full_name} уже зареєстрований у групі {student.group}")
        self._students.append(student)
        return student

    def all(self) -> list[Student]:
        return list(self._students)

    def groups(self) -> list[str]:
        return sorted({s.group for s in self._students})

    def by_group(self, group: str) -> list[Student]:
        wanted = group.strip().casefold()
        return [s for s in self._students if s.group.casefold() == wanted]

    def best(self, group: str | None = None) -> Student:
        """Top student overall, or inside one group."""
        candidates = self._students if group is None else self.by_group(group)
        if not candidates:
            raise StudentNotFoundError("немає студентів" if group is None else f"у групі {group} немає
```

```

студентів")
    return max(candidates, key=lambda s: s.average_grade)

def group_average(self, group: str) -> float:
    members = self.by_group(group)
    if not members:
        raise StudentNotFoundError(f"у групі {group} немає студентів")
    return mean(s.average_grade for s in members)

def sorted_by_grade(self, descending: bool = True) -> list[Student]:
    """Rating. Sort is stable, so equal grades keep alphabetical order."""
    alphabetical = sorted(self._students, key=lambda s: s.full_name)
    return sorted(alphabetical, key=lambda s: s.average_grade, reverse=descending)

def search(self, *, last_name: str | None = None, group: str | None = None,
            min_grade: float | None = None) -> list[Student]:
    """Multi-criteria search; every criterion given narrows the result."""
    result = self._students
    if last_name is not None:
        prefix = last_name.strip().casefold()
        result = [s for s in result if s.last_name.casefold().startswith(prefix)]
    if group is not None:
        wanted = group.strip().casefold()
        result = [s for s in result if s.group.casefold() == wanted]
    if min_grade is not None:
        result = [s for s in result if s.average_grade >= min_grade]
    return list(result)

def statistics(self) -> dict[str, float]:
    if not self._students:
        return {"count": 0, "mean": 0.0, "min": 0.0, "max": 0.0, "excellent": 0}
    grades = [s.average_grade for s in self._students]
    return {
        "count": len(grades),
        "mean": mean(grades),
        "min": min(grades),
        "max": max(grades),
        "excellent": sum(s.is_excellent for s in self._students),
    }

```

4.5. cli.py

Консольний шар. `format_students()` формує таблицю рядком (а не друкує її), тому її можна тестувати. `parse_grade()` приймає і кому, і крапку як десятковий роздільник. Функція `run_menu()` отримує `ask` — функцію читання рядка — як параметр за замовчуванням `input`; у звіті нижче саме через це меню «проходить» сценарій без клавіатури. Помилки предметної області та некоректне введення перехоплюються всередині циклу, програма не падає.

```

HEADER = f"{'Студент':<26}{ 'Група':<10}{ 'Середній бал':>13}"
RULE = "-" * len(HEADER)
MENU = """
1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи

```

- 6. Рейтинг
- 7. Пошук
- 0. Вихід""

```
def format_students(students: list[Student], title: str) -> str:
    if not students:
        return f"{title}\n{RULE}\n(порожньо)"
    rows = [f"{s.full_name:<26}{s.group:<10}{s.average_grade:>13.2f}" for s in students]
    return "\n".join([title, HEADER, RULE, *rows])

def parse_grade(text: str) -> float:
    """'87,5' and '87.5' are both fine; anything else is a ValueError with a readable message."""
    try:
        return float(text.strip().replace(",", "."))
    except ValueError:
        raise ValueError(f"'{text}' не є числом") from None

actions: dict[str, Callable[[], None]] = {
    "1": show_all, "2": add, "3": show_group, "4": show_best,
    "5": show_average, "6": show_rating, "7": search,
}
while True:
    print(MENU)
    choice = ask("Команда: ").strip()
    if choice == "0":
        print("До побачення.")
        return
    action = actions.get(choice)
    if action is None:
        print("Невідома команда.")
        continue
    try:
        action()
    except StudentManagerError as error:
        print(f"Помилка: {error}")
    except ValueError as error:
        print(f"Некоректні дані: {error}")
```

4.6. main.py

Точка входу. За замовчуванням виконується детермінована демонстрація на вбудованих даних (зручно для звіту і для перевірки), з прапорцем `--menu` запускається інтерактивне меню, `--version` бере номер версії з пакета. Розбір аргументів — стандартний `argparse`.

```
def demo_registry() -> StudentRegistry:
    return StudentRegistry([
        Student("Марта", "Гнатишин", "ФЕП-31с", 93.4),
        Student("Остан", "Дзюба", "ФЕП-31с", 78.9),
        Student("Соломія", "Кравець", "ФЕП-32", 88.1),
        Student("Тарас", "Вовк", "ФЕП-31с", 85.0),
        Student("Ірина", "Пасічник", "ФЕП-32", 96.7),
        Student("Юрій", "Скиба", "ФЕП-33", 71.2),
    ])
```

```

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="student-manager", description="Облік студентів")
    parser.add_argument("--menu", action="store_true", help="інтерактивне меню замість демонстрації")
    parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    return parser

def main(argv: list[str] | None = None) -> None:
    if isinstance(sys.stdout, io.TextIOWrapper) and sys.stdout.encoding.lower() != "utf-8":
        sys.stdout.reconfigure(encoding="utf-8")
    args = build_parser().parse_args(argv)
    registry = demo_registry()
    if args.menu:
        run_menu(registry)
    else:
        run_demo(registry)

if __name__ == "__main__":
    main()

```

4.7. tests/test_registry.py

Дев'ять pytest-тестів покривають фільтрацію за групою, середній бал, найкращого студента, валідацію оцінки та назви групи, відхилення дублікатів, пошук за кількома критеріями і порядок рейтингу. Тестується лише бізнес-логіка — саме тому вона й відокремлена від меню.

```

import pytest

from student_manager.exceptions import InvalidGradeError, InvalidGroupError, StudentNotFoundError
from student_manager.models import Student
from student_manager.registry import StudentRegistry

@pytest.fixture
def registry() -> StudentRegistry:
    return StudentRegistry([
        Student("Марта", "Гнатишин", "ФЕП-31с", 93.4),
        Student("Остан", "Дзюба", "ФЕП-31с", 78.9),
        Student("Соломія", "Кравець", "ФЕП-32", 88.1),
    ])

def test_by_group_ignores_case_and_spaces(registry: StudentRegistry) -> None:
    assert [s.last_name for s in registry.by_group(" феП-31с ")] == ["Гнатишин", "Дзюба"]

def test_group_average(registry: StudentRegistry) -> None:
    assert registry.group_average("ФЕП-31с") == pytest.approx(86.15)

def test_best_overall_and_inside_group(registry: StudentRegistry) -> None:
    assert registry.best().last_name == "Гнатишин"

```

```

assert registry.best("ФЕП-32").last_name == "Кравець"

def test_best_of_unknown_group_raises(registry: StudentRegistry) -> None:
    with pytest.raises(StudentNotFoundError):
        registry.best("ФЕП-99")

def test_grade_outside_scale_is_rejected() -> None:
    with pytest.raises(InvalidGradeError):
        Student("Ім'я", "Прізвище", "ФЕП-31с", 101.0)

def test_group_name_is_validated() -> None:
    with pytest.raises(InvalidGroupError):
        Student("Ім'я", "Прізвище", "група 31", 50.0)

def test_duplicate_student_is_rejected(registry: StudentRegistry) -> None:
    with pytest.raises(ValueError):
        registry.add(Student("Марта", "Гнатишин", "ФЕП-31с", 50.0))

def test_search_combines_criteria(registry: StudentRegistry) -> None:
    found = registry.search(group="ФЕП-31с", min_grade=80)
    assert [s.last_name for s in found] == ["Гнатишин"]

def test_rating_is_descending(registry: StudentRegistry) -> None:
    grades = [s.average_grade for s in registry.sorted_by_grade()]
    assert grades == sorted(grades, reverse=True)

```

5. Файл pyproject.toml

[build-system] вказує, що проєкт збирає setuptools. [project] — метадані: назва, версія, мінімальна версія Python 3.11 (потрібна для синтаксису X | None і slots=True), автор, порожній список залежностей (проєкт працює на стандартній бібліотеці). pytest винесено в необов'язкову групу test. [project.scripts] створює команду student-manager, [tool.setuptools.packages.find] каже шукати пакети в src, а [tool.pytest.ini_options] додає src до шляху імпорту для тестів.

```

[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "student-manager"
version = "0.1.0"
description = "Облік студентів – наскрізний навчальний проєкт курсу «Професійний Python»"
readme = "README.md"
requires-python = ">=3.11"

```

```

authors = [{ name = "Artur Lys", email = "Artur.Lys@lnu.edu.ua" }]
dependencies = []

[project.optional-dependencies]
test = ["pytest>=8.0"]

[project.scripts]
student-manager = "student_manager.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

```

6. Файл .gitignore

До репозиторію не потрапляють віртуальне середовище, байт-код і кеші інструментів, артефакти збірки, налаштування IDE та файл .env із секретами.

```

# virtual environment
.venv/

# caches and byte-code
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.ruff_cache/

# build artifacts
dist/
build/
*.egg-info/

# IDE
.idea/
.vscode/

# local secrets
.env

```

7. Фрагмент README.md

Student Manager

Наскрізний навчальний проєкт курсу «Професійний Python» (ЛНУ ім. Івана Франка, ФЕП-31с).
Варіант 1 – система обліку студентів: ім'я, прізвище, група, середній бал.

Лабораторна робота №1: структура проєкту (src-layout), модель даних, бізнес-логіка, консольне меню, тести, Git.

```
## Вимоги
```

```
Python 3.11+
```

```
## Встановлення
```

```
```
```

```
python -m venv .venv
.venv\Scripts\activate # Linux/macOS: source .venv/bin/activate
python -m pip install -e ".[test]"
```
```

```
## Запуск
```

```
```
```

```
python -m student_manager.main # демонстрація на вбудованих даних
python -m student_manager.main --menu # інтерактивне меню
student-manager --version # console script з pyproject.toml
```
```

```
## Тести
```

```
```
```

```
python -m pytest
```
```

8. Створення virtual environment та запуск

Послідовність команд у корені проєкту (Windows):

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -e ".[test]"
python -m student_manager.main
python -m student_manager.main --menu
```

Перевірка інтерпретатора, console script та списку встановлених пакетів у середовищі:

```
$ python --version
Python 3.13.0
$ student-manager --version
student-manager 0.1.0
$ python -m pip list
Package          Version Editable project location
-----
colorama         0.4.6
iniconfig        2.3.0
packaging        26.3
pip              24.2
pluggy          1.6.0
Pygments         2.21.0
pytest          9.1.1
student-manager  0.1.0  C:\Users\user\vscode\uni\python\project
```

9. Результат роботи програми

9.1. Демонстраційний режим

Вивід команди `python -m student_manager.main`. Останній блок показує, як відхиляється спроба додати студента з балом поза шкалою.

УСІ СТУДЕНТИ		
Студент	Група	Середній бал

Гнатишин Марта	ФЕП-31с	93.40
Дзюба Остап	ФЕП-31с	78.90
Кравець Соломія	ФЕП-32	88.10
Вовк Тарас	ФЕП-31с	85.00
Пасічник Ірина	ФЕП-32	96.70
Скиба Юрій	ФЕП-33	71.20

ГРУПА ФЕП-31с		
Студент	Група	Середній бал

Гнатишин Марта	ФЕП-31с	93.40
Дзюба Остап	ФЕП-31с	78.90
Вовк Тарас	ФЕП-31с	85.00

Середній бал групи ФЕП-31с: 85.77
Найкращий студент: Пасічник Ірина (ФЕП-32, 96.70)
Найкращий у ФЕП-31с: Гнатишин Марта (93.40)

РЕЙТИНГ		
Студент	Група	Середній бал

Пасічник Ірина	ФЕП-32	96.70
Гнатишин Марта	ФЕП-31с	93.40
Кравець Соломія	ФЕП-32	88.10
Вовк Тарас	ФЕП-31с	85.00
Дзюба Остап	ФЕП-31с	78.90
Скиба Юрій	ФЕП-33	71.20

ПОШУК: група ФЕП-32, бал >= 90		
Студент	Група	Середній бал

Пасічник Ірина	ФЕП-32	96.70

Статистика: 6 студентів, середній бал 85.55, min 71.2, max 96.7, відмінників 2

Спроба додати студента з балом 120:
відхилено – середній бал має бути в межах 0..100, отримано 120.0

9.2. Інтерактивне меню

Сеанс роботи з меню (`--menu`): перегляд, додавання студента з комою в балі, середній бал нової групи, пошук за префіксом прізвища та мінімальним балом, а також три помилкові дії — нечислова оцінка, неіснуюча група, невідома команда.

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг
7. Пошук
0. Вихід

Команда: 1

УСІ СТУДЕНТИ

Студент	Група	Середній бал

Гнатишин Марта	ФЕП-31с	93.40
Дзюба Остап	ФЕП-31с	78.90
Кравець Соломія	ФЕП-32	88.10
Вовк Тарас	ФЕП-31с	85.00
Пасічник Ірина	ФЕП-32	96.70
Скиба Юрій	ФЕП-33	71.20

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг
7. Пошук
0. Вихід

Команда: 3

Група: ФЕП-32

ГРУПА ФЕП-32

Студент	Група	Середній бал

Кравець Соломія	ФЕП-32	88.10
Пасічник Ірина	ФЕП-32	96.70

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг
7. Пошук
0. Вихід

Команда: 2

Ім'я: Назар

Прізвище: Ковальчук

Група: ФЕП-33

Середній бал: 82,5

Додано: Ковальчук Назар, ФЕП-33, 82.50

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг

7. Пошук
0. Вихід
Команда: 5
Група: ФЕП-33
Середній бал групи ФЕП-33: 76.85

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг
7. Пошук

0. Вихід
Команда: 7
Прізвище (Enter – будь-яке): К
Група (Enter – будь-яка):
Мінімальний бал (Enter – без обмеження): 80

ПОШУК

Студент	Група	Середній бал
Кравець Соломія	ФЕП-32	88.10
Ковальчук Назар	ФЕП-33	82.50

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг

7. Пошук
0. Вихід
Команда: 2
Ім'я: Богдан
Прізвище: Тимків
Група: ФЕП-31с
Середній бал: abc
Некоректні дані: 'abc' не є числом

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг

7. Пошук
0. Вихід
Команда: 5
Група: ФЕП-99
Помилка: у групі ФЕП-99 немає студентів

1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг

```
7. Пошук
0. Вихід
Команда: 9
Невідома команда.
```

```
1. Усі студенти
2. Додати студента
3. Студенти групи
4. Найкращий студент
5. Середній бал групи
6. Рейтинг
7. Пошук
0. Вихід
Команда: 0
До побачення.
```

10. Автоматизовані тести

```
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\user\vscode\uni\python\project\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\vscode\uni\python\project
configfile: pyproject.toml
testpaths: tests
collecting ... collected 9 items

tests/test_registry.py::test_by_group_ignores_case_and_spaces PASSED [ 11%]
tests/test_registry.py::test_group_average PASSED [ 22%]
tests/test_registry.py::test_best_overall_and_inside_group PASSED [ 33%]
tests/test_registry.py::test_best_of_unknown_group_raises PASSED [ 44%]
tests/test_registry.py::test_grade_outside_scale_is_rejected PASSED [ 55%]
tests/test_registry.py::test_group_name_is_validated PASSED [ 66%]
tests/test_registry.py::test_duplicate_student_is_rejected PASSED [ 77%]
tests/test_registry.py::test_search_combines_criteria PASSED [ 88%]
tests/test_registry.py::test_rating_is_descending PASSED [100%]

===== 9 passed in 0.09s =====
```

11. Робота з Git

Репозиторій ініціалізовано на гілці main. Історія складається з шести комітів, кожен відповідає логічному етапу: каркас, модель і винятки, бізнес-логіка, консоль і точка входу, тести з README, дрібне виправлення після перевірки туру --strict.

```
git init -b main
git add .gitignore pyproject.toml src/student_manager/__init__.py src/student_manager/config.py
git commit -m "Create project skeleton with src-layout"

git add src/student_manager/exceptions.py src/student_manager/models.py
git commit -m "Add Student model with validation and domain exceptions"

git add src/student_manager/registry.py
git commit -m "Add StudentRegistry business logic"
```

```
git add src/student_manager/cli.py src/student_manager/main.py
git commit -m "Add console menu and application entry point"
```

```
git add tests/test_registry.py README.md
git commit -m "Add registry tests and README"
```

11.1. Результат `git status`

On branch main
nothing to commit, working tree clean

11.2. Результат `git log --oneline`

```
ce3e20c Make stdout re-encoding type-safe for mpy --strict
ca3ca1a Add registry tests and README
c01835d Add console menu and application entry point
8502b7c Add StudentRegistry business logic
7726d17 Add Student model with validation and domain exceptions
9cee0a2 Create project skeleton with src-layout
```

12. Відповіді на контрольні запитання

1. Що таке Python-проект?

Набір модулів, конфігурації, тестів і документації, який встановлюється та запускається як єдине ціле, а не як окремий файл.

2. Чим проєкт відрізняється від одного Python script?

Скрипт — один файл без структури й залежностей. У проєкті є пакет із кількох модулів, `pyproject.toml` з метаданими, тести, README та історія змін у Git.

3. Що таке Python module?

Файл `.py`. Після імпорту він стає об'єктом `module` з власним простором імен.

4. Що таке Python package?

Каталог із модулями (зазвичай з `__init__.py`), який імпортується як одне ім'я: `student_manager.registry`.

5. Для чого потрібний `__init__.py`?

Позначає каталог як пакет і виконується під час першого імпорту; зручне місце для `__version__` та публічного API пакета.

6. Що таке `src-layout`?

Розміщення коду в `src/<пакет>`, а не в корені репозиторію.

7. Які переваги має `src-layout`?

Тести й запуск працюють лише з установленим пакетом, тому неможливо випадково імпортувати код з поточного каталогу; корінь репозиторію не потрапляє в `sys.path`; у wheel потрапляє тільки вміст `src`.

8. Для чого використовується `virtual environment`?

Щоб залежності одного проєкту не конфліктували з іншими проєктами та із системним Python.

9. Як створити `.venv`?

```
python -m venv .venv
```

10. Як активувати `virtual environment`?

Windows: `.venv\Scripts\activate`; Linux/macOS: `source .venv/bin/activate`. Можна й не активувувати, а викликати `.venv\Scripts\python` напямую.

11. Для чого використовується `pip`?

Для встановлення, оновлення та видалення пакетів із PyPI або з локального каталогу.

12. Чому рекомендується `python -m pip`?

Так `pip` гарантовано належить саме тому інтерпретатору, який запущено, а не іншому Python з PATH.

13. Для чого потрібний `pyproject.toml`?

Єдиний файл метаданих і конфігурації: система збірки, назва, версія, залежності, console scripts, налаштування інструментів на кшталт `pytest`.

14. Що містить секція `[project]`?

`name`, `version`, `description`, `readme`, `requires-python`, `authors`, `dependencies`, `optional-dependencies`, `scripts`.

15. Що таке `editable installation`?

`pip install -e .` — у `site-packages` створюється посилання на `src`, тому зміни коду видно одразу, без перевстановлення.

16. Для чого використовується `if __name__ == "__main__":`?

Код під цією умовою виконується лише при прямому запуску файла; під час імпорту (наприклад, у тестах) — ні.

17. Що таке `Git`?

Розподілена система контролю версій: кожен клон містить повну історію.

18. Що таке repository?

Каталог проекту разом із прихованим каталогом `.git`, де зберігаються всі коміти, гілки та налаштування.

19. Що таке commit?

Знімок стану файлів із повідомленням, автором, датою та хешем; посилається на батьківський коміт.

20. Для чого використовується git status?

Показує, які файли змінені, підготовлені до коміту або ще не відстежуються.

21. Для чого git add?

Переносить зміни до staging area, формуючи вміст наступного коміту.

22. Для чого git commit?

Фіксує staging area як новий коміт в історії.

23. Що показує git log?

Історію комітів. З `--oneline` — по одному рядку: скорочений хеш і повідомлення.

24. Для чого потрібен .gitignore?

Перелік шаблонів файлів, які Git не пропонує до коміту: середовище, кеші, збірки, секрети.

25. Чому .venv не потрібно комітити?

Це десятки мегабайт бінарних файлів, прив'язаних до ОС і версії Python; воно відтворюється однією командою з `pyproject.toml`.

26. Для чого потрібний README?

Це перше, що читають у репозиторії: що це за проект, як його встановити, запустити й протестувати.

27. Що таке type hints?

Анотації типів параметрів, значень, що повертаються, та змінних. Інтерпретатор їх не перевіряє, але `mypy`, `Pyright` та IDE — так.

28. Для чого програму розділяють на modules?

Щоб кожен модуль відповідав за одне: модель, логіку, інтерфейс. Так код легше читати, тестувати й змінювати незалежно.

29. Що таке business logic?

Правила предметної області: як обчислюється середній бал групи, хто «найкращий», що вважається дублікатом.

30. Чому потрібно відокремлювати application entry point від business logic?

Логіку тоді можна викликати з тестів, з меню, з майбутнього API без print() та input(); точка входу лише збирає компоненти до купи.

13. Висновок

У роботі створено базовий проєкт «Система обліку студентів» за src-layout: пакет із шести модулів, dataclass-модель із валідацією, власні винятки, клас бізнес-логіки StudentRegistry, консольне меню з обробкою помилок і детермінований демонстраційний режим. Проєкт встановлюється в editable-режимі, має console script, дев'ять pytest-тестів, проходить перевірку типу --strict. Підготовлено pyproject.toml, .gitignore, README.md та історію з шести логічних комітів на гілці main. Цей репозиторій стане основою для наступних лабораторних робіт курсу.